Agentic financial statement generation: architecture

Scope: trial balance + chart of accounts + manual adjustments in; Balance Sheet, P&L, Cash Flow and Statement of Changes in Equity out. **Prototype slice:** the manual-adjustments agent (see section 9 and the repo README).

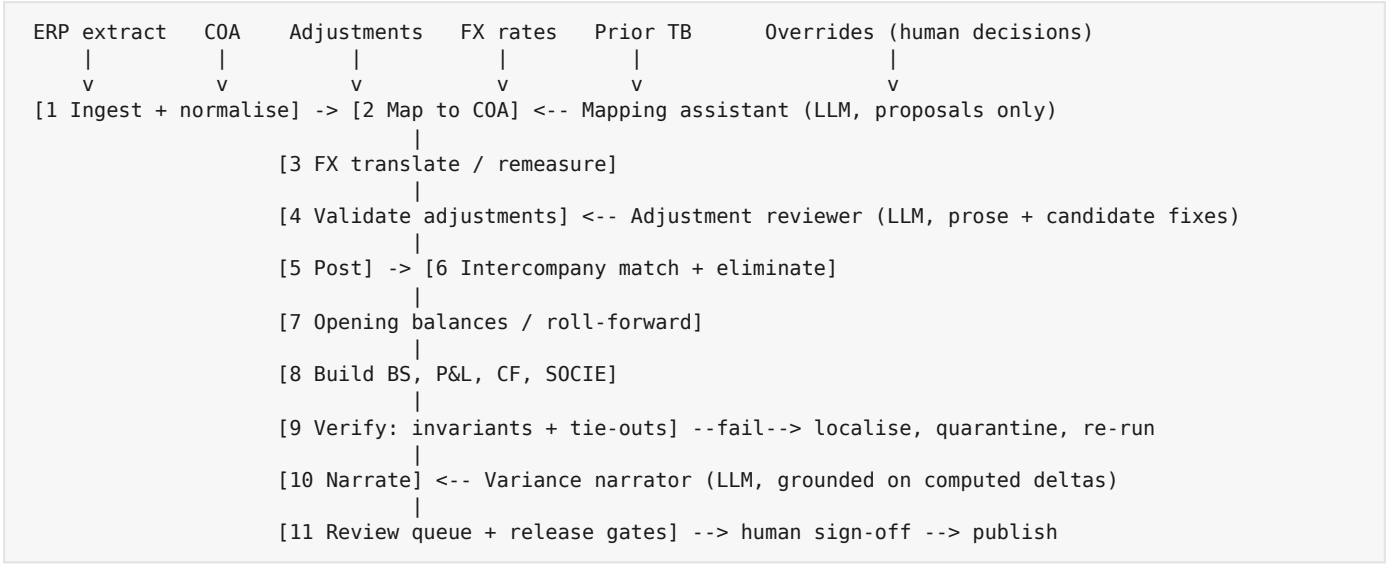
---

1. The short version

Most of this problem is ledger arithmetic, and ledger arithmetic should be plain, deterministic, tested code. The system is a **deterministic pipeline (a state machine in code) that calls three narrowly scoped LLM workers** at the points where the input is ambiguous, unstructured or needs explaining to a person:

1. **Mapping assistant:** proposes a COA node for accounts the rules cannot map.
2. **Adjustment reviewer:** explains why a journal entry was held back and drafts candidate repairs.
3. **Variance narrator:** writes commentary for the statements and the reconciliations.

An LLM never produces a number that lands on a statement and never makes a posting decision. It produces **proposals and prose**. Code validates every proposal before a human sees it, and a human approves anything that changes the books. Every statement cell can be traced back through a lineage graph to a source file, row and content hash.



Every arrow is a typed, versioned artifact on disk or in a store. Each stage is a pure function of its inputs plus the policy version, so re-running with the same inputs gives byte-identical outputs.

---

2. Problem decomposition

“Generate financials” is really eleven sub-problems, and they need different levels of reliability. The table drives every design choice that follows.

#	Sub-problem	Failure cost	Done by	LLM role
1	Ingest and normalise the ERP extract (types, signs,	High, silent	Code	None

#	Sub-problem	Failure cost	Done by	LLM role
	currencies, duplicates)			
2	Map ERP accounts to COA nodes	Very high (misstatement)	Code rules, then mapping memory, then LLM proposal, then human	Proposes for the residue; never auto-applies a first-time mapping
3	FX translation and remeasurement	High	Code, from a policy file a human owns	None
4	Validate manual adjustments	High	Code rules	Explains failures; drafts repairs that code re-validates
5	Post to the ledger	Very high	Code	None
6	Intercompany matching and elimination	High	Code matching on counterparty, amount and date	Fuzzy matching of descriptions as a suggestion
7	Opening balances, roll-forward and restatements	High	Code	Maps renamed or recoded accounts across periods (a proposal)
8	Assemble the statements	Medium (pure arithmetic once 1 to 7 are right)	Code + templates	None
9	Verification and tie-outs	Gatekeeper	Code	None
10	Commentary and variance explanation	Low (advisory)	LLM, grounded	Main job
11	Review, approval, release	Accountability	Human	Summarises the queue

The point of the table: **the LLM earns its keep on rows 2, 4, 6, 7 and 10**, where the inputs are names, memos, descriptions and intent. On rows 3, 5, 8 and 9 an LLM would only add variance and remove auditability.

---

3. Agent topology and why

Choice: an orchestrator in code, plus bounded LLM workers. Not a single tool-using agent, and not a swarm.

Why not a single agent with tools? If an LLM controls the flow, it can skip a step, run steps out of order, or "fix" a failing check by changing an input. Every run would be a different program, and an auditor cannot sign off on that. Financial close is a fixed procedure with known dependencies. That is a DAG, and a DAG should be code.

Why not an orchestrator with many autonomous sub-agents (Mapper, Adjuster, Builder, Validator...)? The Builder and Validator do arithmetic and should not be agents at all. Making them agents adds failure modes (non-determinism, prompt drift, cost) and no capability. The brief rightly calls a 12-agent swarm for a 200-line script a red flag.

Where real agentic behaviour does belong: the **mapping assistant** benefits from a short investigative loop. With read-only tools it can look up the account in prior-period mappings, search the COA by name and type, read GL detail for the account, check the balance's sign and size, and then propose a mapping with cited evidence. It gets a step budget (say 6 tool calls), read-only access, and one structured output type. The adjustment reviewer and the narrator are single-shot calls with schema-constrained output and a retry when validation fails.

Contract for every LLM worker:

- Input: a structured payload that code built, never raw files.
- Output: JSON against a schema. Enumerations come from data (for example, the account must be a code in the COA).
- Validation: code checks the output (schema, enumerations, a whitelist of allowed numbers, consistency with the status code already decided). On failure, one retry with the violations fed back, then a deterministic fallback or escalation to a human.
- Trace: prompt version, model id, input hash, raw output and validation result are all logged.

---

4. Where the line sits between code and LLM

Rule: if a step's output is a number, a balance, a status or a posting, code produces it. If the output is a choice among candidates under ambiguity, or an explanation, an LLM may draft it, code checks it, and a human owns it.

Decision	Code	LLM	Human
Does this JE balance? Is the account valid? Is the date in period?	Yes		
Is the FX revaluation amount right?	Recomputes it from the rate table		
Is this entry circular or self-cancelling?	Graph and netting checks		
Which COA node does "Suspense - Unmapped" belong to?	Rules + mapping memory	Ranked proposals with evidence	Approves
Which side of an unbalanced reclass is the typo?	Generates both balanced candidates	Can rank them using the source document text when available	Picks one
Explain to a controller why JE-008 was rejected	Template fallback	Plain-English draft	Reads it
Is 850,000 of bonus accrual reasonable?	Materiality flag	Could compare to history as context	Signs off
FX policy (which rate, fallback rules)	Applies the policy		Owns and versions it

The LLM is valuable on the **long tail of semantic ambiguity**: abbreviations ("Conf travel"), renamed accounts across ERPs and periods ("Sundry Operating Expenses" 6905 became "Other Operating Expenses" 6900), memo text that contradicts the account used, and turning twenty machine finding codes into four sentences a finance user will act on. That is real value. It is just not arithmetic.

---

5. Failure modes that kill you in production

Each failure mode below gets detection, a response and an owner. In brackets is where the provided data triggers it.

5.1 Hallucinated account mappings

- **Constrain:** the model can only output a code from the COA enumeration and must cite evidence (name similarity, prior-period mapping, sign and size of the balance, GL memos).
- **Verify:** code re-checks the proposal: the node is postable, statement type and normal balance are consistent with the sign of the balance, and the mapping does not create a circular or self-cancelling entry. In the prototype, the nearest-name match for 6315 is 6310, which would turn JE-005 into a debit and credit to the same account. The candidate is discarded on re-validation and the discard is shown to the reviewer.
- **Calibrate:** confidence is calibrated against the history of human decisions, not taken from the model's own guess. Below a threshold, or above a materiality level, the mapping goes to a human. The first mapping for a new ERP account always does.
- **Remember:** approved mappings go into a versioned mapping memory keyed by (ERP, entity, account code, name hash). Next period they are deterministic, and the LLM only sees what is new or changed. Mapping drift (same code, new name) re-opens the mapping.

5.2 Debits != credits after adjustments

- Per entry: a hard rule. An unbalanced entry never posts and cannot be waived by an override (JE-002, out by 3,500).
- Per batch: after posting, the net of the TB must not move (invariant I2). Any change means a bug, and the batch is stopped.
- Per TB: the pre-adjustment TB is checked in the functional currency. The supplied TB is out by **182,460.20 USD after translation** (and -4,800.00 if currencies are summed as-is). That is not rounding. The likely cause is that the extract has no functional-currency book values for the EUR/GBP rows, so translating them at the closing rate builds in the unrecorded remeasurement. This is clarifying question 1.
- **Never plug to suspense automatically.** The prototype lists "plug to 9999" as an option it considered and ruled out.

5.3 An account that fits no COA node

- Orphan accounts (9999 Suspense, 12,400; 6315 in JE-005) are kept, never dropped, and shown on a visible "Unmapped" line.
- **Release gate:** statements cannot be published while unmapped balances exceed a threshold (default: any non-zero amount).
- Resolution: map to an existing node (proposal + approval) or raise a COA change request owned by the COA owner. The prototype supports the second path: approving it adds the account and records the change request in the run output.
- Structural COA problems are flagged at load time: a header with no children (1290), a postable account with children (8000, 3300, which can double count in rollups), a header with no normal balance (7000), and cash-flow category TBD (1150, 2170).

5.4 FX gaps

- The rate policy is explicit and versioned: BS at period-end, P&L at period-average, equity at historical, with a declared fallback chain.
- A missing rate (GBP period-end) uses the fallback **and** raises a finding that blocks release until someone signs it off. Silent interpolation is never allowed.
- Remeasurement vs translation is a policy question, not a code question. Foreign-currency cash held by a USD-functional entity is remeasured through P&L. A foreign subsidiary's net assets are translated through CTA (3310). The data hints at both, so I asked.
- A manual FX revaluation entry is recomputed from the rate file. JE-003 posts 11,200. Average to period-end gives 10,730.20 and opening to period-end gives 19,809.60, so neither matches. If the pipeline already remeasures EUR cash, posting JE-003 would also double count.
- Equity needs historical rates, which the bundle does not include.

5.5 Circular intercompany entries

- Within an entry: per-account netting. Every account nets to zero, so the entry has no effect (JE-008: 2170 debited and credited 320,000). It is rejected, because posting it would add a 320,000 "settlement" to the audit trail that moved nothing.
- Across entities: IC balances form a graph (entity A owes B). Cycles and asymmetric balances (A says it owes B 100, B says B is owed 90) are found with matching rules on counterparty, amount and date. Unmatched balances go to an IC reconciliation queue before elimination.
- 2170 has no IC receivable on the other side in this entity, and no counterparty field anywhere, so elimination is impossible without the other entity's books.

5.6 Other defects in the bundle that the design has to absorb

- **Duplicate TB row:** 6310 appears twice (245,000 and 38,500). The second row could be a double export or a legitimate subledger split. Both rows are kept and flagged, and the adjustments that touch 6310 (JE-002, JE-005) carry a warning.
- **Prior-period rename:** 6905 "Sundry Operating Expenses" (prior) vs 6900 "Other Operating Expenses" (current). The code and the name both changed. Comparatives need a cross-period mapping (a mapping assistant job, approved by a human).
- **Prior TB is not a usable opening balance:** it holds balance-sheet accounts only plus one stray P&L account, and it is out of balance by 2,832,800 on raw amounts.
- **Retained earnings does not roll:** prior 5,180,000 vs current 7,240,000 before closing current-year income (about 6.0M). The 2,060,000 gap needs an explanation (dividends, a restatement, an interim close) before the SOCIE can be built.
- **Gains in expense-typed accounts:** 7310 and 7400 carry credit balances. Sign handling must follow the balance, not the account type label.
- **Cash flow inputs are thin:** PP&E cost rose from 11.8M to 17.0M with no capex, disposal or lease detail. An indirect-method CF built from balance deltas alone will be wrong (non-cash additions, FX effect on cash, debt reclasses).

---

6. Validation and self-correction loop

Checks run in layers. A failure is **localised and contained**, never "retried until it passes".

1. **Load checks:** schema, types, duplicates, orphans, structural COA problems, missing rates. These become findings, and nothing is cleaned silently.
2. **Stage rules:** per-entry and per-batch rules (section 5). Every rule emits a finding with a code, a severity, facts and source refs. Severity decides the status:

    - FATAL: rejected, no safe repair.
    - FIXABLE: quarantined, with candidate repairs.
    - WARNING: accepted, needs sign-off.
    - INFO: context only.
3. **Repair proposals:** code generates candidates. The LLM may rank them or explain them. **Every candidate is re-run through the full rule set**, and only survivors are shown. Nothing is auto-applied.
4. **Post-posting invariants:** checked by an independent recomputation from the decisions, not from the posting routine's own output:
    - I1: every posted entry balances.
    - I2: the batch does not change the TB net.
    - I3: each account's delta equals the sum of the posted lines.
    - I4: every posted line hits a COA account.
    - I5: every adjustment traces to a source line.
    - Plus: does any account flip against its normal balance?
5. **Self-correction:** if an invariant fails, the verifier names the offending entry, demotes it to QUARANTINED with a `POST_INVARIANT_FAILED` finding, and re-runs posting, up to N passes. A batch-level failure with no entry to blame stops the run. A test proves the verifier works independently: it monkey-patches the rules so an unbalanced entry slips through, and the verifier catches it and pulls it back.
6. **Statement tie-outs (full system):**
    - Assets = liabilities + equity.
    - P&L net income = the SOCIE net income line.
    - CF closing cash = BS cash.
    - Opening equity + movements = closing equity.
    - Current comparatives = prior published figures, unless a restatement has been approved.
    - Each tie-out failure names the accounts involved.
7. **LLM output checks:** schema; every blocking finding must be addressed; every number of 100 or more must appear in the input payload (catches invented figures); no wording that contradicts the status (for example "has been posted" on a quarantined entry). One retry with the violations fed back, then the template fallback. Each attempt is traced.
8. **Release gates:** publishing is blocked while there are unmapped balances, unsigned FX fallbacks, quarantined entries above materiality, or any failed tie-out.

Explicit non-goal: the system never lets a model edit inputs or numbers to make a check pass. That is the most dangerous failure in an agentic finance system, because it looks like success.

---

7. How an auditor traces a balance sheet cell

Every number is a node in a **lineage graph**:

```
Statement cell (BS "Accrued expenses", 2024-Q4, run ee681dd4...)
  = line item "Accrued expenses"  <- COA rollup, mapping version m-17
    = account 2120 post-adjustment balance  -2,075,000.00
        = trial_balance.csv#row 19                     -1,150,000.00   (sha256 6a5d38f4..., rate 1, functional)
        + manual_adjustments.json#entries[0].lines[1]    -850,000.00   (JE-001, accepted, signed off by <reviewe
        + manual_adjustments.json#entries[8].lines[1]     -75,000.00   (JE-009, accepted)
```

That is the actual `post_adjustment_tb.csv` row for 2120 from the prototype run. Design rules:

- **Every value carries a source ref:** file, row or JSON path, and the file's sha256. Translated values also carry the rate and its basis (for example, "period_average (FALLBACK: period_end missing)").
- **Append-only, content-addressed runs:** run id = hash(input hashes + policy version + overrides + code hash). The same inputs give the same run id and byte-identical outputs (there is a test for this). An auditor can re-run any historical close.
- **Human decisions are inputs, not side effects:** overrides live in a file (later, a table) with approver, reason and timestamp. The file is hashed into the run, so a changed decision means a new run with a visible diff.
- **The trace log** (`trace.jsonl`) records every finding, proposed and discarded fix, override, LLM attempt and verification pass in order.
- **Drill-down UI (full product):** click a cell to see the accounts, then the ledger lines, then the source rows, approvals and rule results, with "why is this here?" answered by the trace.

---

8. Human in the loop and overrides

- **Roles:** preparer (submits entries), reviewer (signs off warnings, picks fixes), COA owner (approves new accounts and mappings), controller (releases).
- **Allowed actions:** `sign_off`, `apply_fix` (only a proposal that passed re-validation), `accept_as_is` (only for judgment-call findings, each acknowledged by name), `confirm_reject`.
- **Hard rules cannot be waived by anyone:** balance, account existence, no-effect entries, suspense plugs. The prototype refuses an override that tries, and shows the refusal in the report.
- **Every override is re-validated before posting.** An approver cannot post an unbalanced entry by mistake.
- Next steps for the full product: four-eyes (approver must not be the preparer), role-based permissions, and expiry of sign-offs when the underlying numbers change.

---

9. The prototype slice: manual-adjustments agent

What it does: loads the five files unchanged with provenance; translates the TB with a flagged GBP fallback; runs 12 rule families across each entry and the batch; classifies each entry; generates and re-validates repairs; applies human overrides; writes plain-English explanations (template by default, Claude with guardrails via `--llm`); posts accepted entries; verifies invariants with self-correction; and writes the report, decisions, post-adjustment TB with lineage, review queue, manifest and trace.

Results on the supplied data:

Entry	Decision	Why
JE-001	Accepted, needs sign-off	850,000 is above materiality (340,325 = 5% of pre-tax income)
JE-002	Quarantined	Debits 28,500 vs credits 25,000; two balanced options, no suspense plug
JE-003	Quarantined	11,200 matches no recomputation (10,730.20 / 19,809.60); would double count if the pipeline remeasures
JE-004, 006, 007, 009, 010	Accepted	All checks pass

Entry	Decision	Why
JE-005	Quarantined	6315 not in COA; mapping to 6310 discarded as circular; COA-change option offered
JE-008	Rejected	Debit and credit to 2170 for 320,000: no effect, no counterparty

Deliberately out of scope: TB-to-COA mapping, IC elimination, statement assembly and tie-outs. Their inputs show up as “upstream data issues” in the report rather than being fixed quietly.

---

10. Assumptions made pending answers (see CLARIFYING_QUESTIONS.md)

1. Foreign TB rows are in transaction currency (the input README says to translate them). The pipeline remeasures monetary BS items at period-end, so manual FX reval entries are treated as duplicates unless a human decides otherwise.
2. Single entity (USD functional). Intercompany entries need a counterparty to be eliminable.
3. Materiality is 5% of pre-tax income for sign-off purposes. This is a policy value, and the controller should set it.
4. The adjustment batch is unposted and all-or-nothing per entry. No partial posting of an entry’s lines.